

Here's a **comprehensive list of 120 MCQs** (10 for each topic) on Java and Data Structures, with detailed answers and explanations.

1. Basics of Algorithm (10 Questions)

Q1. What is the key characteristic of a greedy algorithm?

- A.** It always produces the optimal solution.
- B.** It explores all possible solutions before choosing the best one.
- C.** It builds up a solution piece by piece, choosing the next piece that offers the most immediate benefit.
- D.** It divides the problem into subproblems and solves them recursively.

✓ **Answer:** C. It builds up a solution piece by piece, choosing the next piece that offers the most immediate benefit.

💡 **Explanation:**

Greedy algorithms make locally optimal choices at each step, hoping to find a global optimum.

Q2. Which of the following is NOT a characteristic of a dynamic programming algorithm?

- A.** Overlapping subproblems
- B.** Optimal substructure
- C.** Greedy choice property
- D.** Memoization

✓ **Answer:** C. Greedy choice property

💡 **Explanation:**

Greedy algorithms and dynamic programming differ in that greedy algorithms do not necessarily guarantee an optimal solution.

Q3. Which algorithm is used in Huffman coding?

- A.** Dynamic Programming
- B.** Divide and Conquer
- C.** Greedy Algorithm
- D.** Backtracking

✓ **Answer:** C. Greedy Algorithm

💡 **Explanation:**

Huffman coding uses a greedy approach to build a prefix-free binary tree for minimum-cost encoding.

Q4. What is the output of the following code?

```
java
CopyEdit
public class Main {
    public static int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }

    public static void main(String[] args) {
        System.out.println(gcd(56, 98));
    }
}
```

- A. 7
- B. 14
- C. 28
- D. 56

✓ **Answer:** B. 14

💡 **Explanation:**

GCD of 56 and 98 is 14 (using Euclidean algorithm).

Q5. Which of the following algorithms follow the **divide and conquer** paradigm?

- A. Merge Sort
- B. Dijkstra's Algorithm
- C. Kruskal's Algorithm
- D. Bellman-Ford Algorithm

✓ **Answer:** A. Merge Sort

💡 **Explanation:**

Divide and conquer divides the problem into smaller subproblems and merges them.

Q6. What is the best case time complexity of Quick Sort?

- A. $O(n^2)$
- B. $O(n \log n)$

- C. $O(n)$
- D. $O(\log n)$

✓ **Answer:** B. $O(n \log n)$

💡 **Explanation:**

Best case occurs when the pivot divides the array into two equal halves.

Q7. Which algorithm is used to solve the **Travelling Salesman Problem**?

- A. Dynamic Programming
- B. Greedy Algorithm
- C. Divide and Conquer
- D. Backtracking

✓ **Answer:** A. Dynamic Programming

💡 **Explanation:**

TSP is solved efficiently using dynamic programming to minimize cost.

Q8. Which technique is used in Kruskal's algorithm?

- A. Dynamic Programming
- B. Greedy Algorithm
- C. Backtracking
- D. Divide and Conquer

✓ **Answer:** B. Greedy Algorithm

💡 **Explanation:**

Kruskal's algorithm builds the minimum spanning tree by choosing the lowest-weight edge.

Q9. Which of the following algorithm paradigms works by solving smaller subproblems first?

- A. Divide and Conquer
- B. Greedy Algorithm
- C. Dynamic Programming
- D. Backtracking

✓ **Answer:** C. Dynamic Programming

💡 **Explanation:**

Dynamic programming uses a bottom-up approach and solves subproblems to build the solution.

Q10. What is the time complexity of the **Euclidean algorithm** for finding GCD?

- A. $O(\log n)$
- B. $O(n \log n)$
- C. $O(n)$
- D. $O(n^2)$

✓ **Answer:** A. $O(\log n)$

💡 **Explanation:**

The Euclidean algorithm computes GCD in logarithmic time.

□ 2. Time Complexity (10 Questions)

Q11. What is the time complexity of a nested loop structure as shown below?

```
java
CopyEdit
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + j);
    }
}
```

- A. $O(n)$
- B. $O(n^2)$
- C. $O(\log n)$
- D. $O(n \log n)$

✓ **Answer:** B. $O(n^2)$

💡 **Explanation:**

The outer and inner loops both iterate n times, giving $O(n^2)$ complexity.

Q12. What is the time complexity of adding an element at the end of an array?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n \log n)$

✓ **Answer:** B. $O(1)$

💡 **Explanation:**

Appending an element takes constant time in arrays.

Q13. What is the time complexity of binary search?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

✓ **Answer:** C. $O(\log n)$

💡 **Explanation:**

Binary search divides the array into halves, taking logarithmic time.

Q14. What is the worst-case time complexity of Quick Sort?

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. $O(\log n)$

✓ **Answer:** C. $O(n^2)$

💡 **Explanation:**

Worst case occurs when the pivot divides the array unevenly.

Q15. What is the time complexity of finding the maximum element in an unsorted array?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

✓ **Answer:** C. $O(n)$

💡 **Explanation:**

Linear search takes $O(n)$ time to find the maximum element.

Q16. What is the time complexity of matrix multiplication for two $n \times n$ matrices?

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. $O(n^3)$

✓ **Answer:** D. $O(n^3)$

💡 **Explanation:**

Matrix multiplication requires n^3 scalar multiplications.

Q17. Which of the following has the worst time complexity?

- A. Merge Sort
- B. Binary Search
- C. Bubble Sort
- D. Insertion Sort

✓ **Answer:** C. Bubble Sort

💡 **Explanation:**

Bubble sort has worst-case complexity of $O(n^2)$.

Q18. What is the best-case time complexity of Insertion Sort?

- A. $O(n \log n)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(1)$

✓ **Answer:** B. $O(n)$

💡 **Explanation:**

Best case occurs when the array is already sorted.

Q19. What is the time complexity to search an element in a HashMap?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

✓ **Answer:** A. $O(1)$

💡 **Explanation:**

HashMap uses hashing, which gives constant time complexity for search operations.

Q20. What is the average case time complexity of merge sort?

- A. $O(n)$
- B. $O(n \log n)$

C. $O(\log n)$

D. $O(n^2)$

✓ **Answer:** B. $O(n \log n)$

💡 **Explanation:**

Merge Sort divides the array into halves and merges them recursively.

📁 3. Sorting (10 Questions)

Q21. Which sorting algorithm works best for nearly sorted arrays?

A. Bubble Sort

B. Quick Sort

C. Insertion Sort

D. Merge Sort

✓ **Answer:** C. Insertion Sort

💡 **Explanation:**

Insertion Sort works efficiently with nearly sorted arrays.

Q22. Which sorting algorithm is not based on comparison?

A. Quick Sort

B. Merge Sort

C. Counting Sort

D. Bubble Sort

✓ **Answer:** C. Counting Sort

💡 **Explanation:**

Counting Sort uses counting and indexing rather than comparison.

Q23. What is the worst-case time complexity of Merge Sort?

A. $O(n \log n)$

B. $O(n^2)$

C. $O(n)$

D. $O(\log n)$

✓ **Answer:** A. $O(n \log n)$

💡 **Explanation:**

Merge Sort recursively divides the array, taking $O(n \log n)$.

Q24. Which of the following algorithms is NOT a stable sort?

- A. Bubble Sort
- B. Merge Sort
- C. Quick Sort
- D. Insertion Sort

✓ **Answer:** C. Quick Sort

💡 **Explanation:**

Quick Sort is not stable by default.

Q25. What is the best case time complexity of Bubble Sort?

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. $O(1)$

✓ **Answer:** A. $O(n)$

💡 **Explanation:**

Best case occurs when the array is already sorted.

Q26. Which algorithm uses a pivot element to partition the array?

- A. Merge Sort
- B. Quick Sort
- C. Bubble Sort
- D. Heap Sort

✓ **Answer:** B. Quick Sort

💡 **Explanation:**

Quick Sort partitions the array based on a pivot.

Q27. Which sorting algorithm is used in Java's `Arrays.sort()` for primitive data types?

- A. Merge Sort
- B. Quick Sort

- C. Dual-Pivot Quick Sort
- D. Bubble Sort

✓ **Answer:** C. Dual-Pivot Quick Sort

💡 **Explanation:**

Java uses an optimized version of Quick Sort for primitive types.

Q28. Which sorting algorithm performs well on small datasets?

- A. Merge Sort
- B. Quick Sort
- C. Insertion Sort
- D. Heap Sort

✓ **Answer:** C. Insertion Sort

💡 **Explanation:**

Insertion Sort has better performance for small datasets.

Q29. Which of the following sorting algorithms uses the concept of heaps?

- A. Merge Sort
- B. Quick Sort
- C. Heap Sort
- D. Insertion Sort

✓ **Answer:** C. Heap Sort

💡 **Explanation:**

Heap Sort uses a binary heap data structure.

Q30. What is the space complexity of Merge Sort?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

✓ **Answer:** A. $O(n)$

💡 **Explanation:**

Merge Sort requires auxiliary space for merging.

🔍 4. Searching (10 Questions)

Q31. What is the time complexity of linear search?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

✓ **Answer:** B. $O(n)$

💡 **Explanation:**

Linear search iterates through the entire array.

Q32. Binary search works best with which type of data?

- A. Unsorted Arrays
- B. Linked Lists
- C. Sorted Arrays
- D. Random Data

✓ **Answer:** C. Sorted Arrays

💡 **Explanation:**

Binary search requires a sorted array.

Q33. What is the best-case time complexity of Binary Search?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

✓ **Answer:** A. $O(1)$

💡 **Explanation:**

Best case occurs when the target element is found at the middle.

Q34. What is the time complexity to search in a balanced Binary Search Tree (BST)?

- A. $O(1)$
- B. $O(\log n)$

- C. $O(n)$
- D. $O(n^2)$

✓ **Answer:** B. $O(\log n)$

💡 **Explanation:**

Balanced BST ensures logarithmic search time.

Q35. What is the time complexity of searching an element in a hash table?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

✓ **Answer:** A. $O(1)$

💡 **Explanation:**

Hash tables provide constant time search using hashing.

Q36. Which of the following is a disadvantage of linear search?

- A. Requires sorted array
- B. Inefficient for large datasets
- C. Complex to implement
- D. Space complexity is high

✓ **Answer:** B. Inefficient for large datasets

💡 **Explanation:**

Linear search performs poorly on large datasets with $O(n)$ complexity.

✓ **[Remaining 80 questions for Array Operations, Stack, Queue, Linked List, Recursion, Infix to Postfix, and Evaluation of Infix to Postfix are coming up...]**

✉ **Let me know if you'd like to proceed with the next sections!** 😊

40

You said:

continue

ChatGPT said:

Here's the **continuation** with 80 more **MCQs on Java and Data Structures:**

5. Array Operations (10 Questions)

Q41. What is the time complexity to access an element in an array?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

✓ **Answer:** A. $O(1)$

💡 **Explanation:**

Accessing an element using an index in an array takes constant time.

Q42. What is the worst-case time complexity to insert an element at the beginning of an array?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

✓ **Answer:** B. $O(n)$

💡 **Explanation:**

Shifting all elements to the right takes linear time.

Q43. What is the time complexity to delete an element from the middle of an array?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

✓ **Answer:** C. $O(n)$

💡 **Explanation:**

Elements after the deleted item must be shifted.

Q44. Which of the following is the correct way to declare a 2D array in Java?

- A. `int arr[][] = new int[3][];`
- B. `int arr[][] = new int[3][3];`
- C. `int arr[3][3];`
- D. `int arr = new int[3][3];`

✓ **Answer:** B. `int arr[][] = new int[3][3];`

💡 **Explanation:**

Arrays in Java are declared using `new`.

Q45. What will be the output of the following code?

```
java
CopyEdit
int[] arr = {1, 2, 3, 4, 5};
System.out.println(arr[3]);
```

- A. 3
- B. 4
- C. 5
- D. `ArrayIndexOutOfBoundsException`

✓ **Answer:** B. 4

💡 **Explanation:**

Array indices start from 0, so `arr[3]` is 4.

Q46. What is the time complexity of finding the minimum element in an unsorted array?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

✓ **Answer:** B. $O(n)$

💡 **Explanation:**

A linear search is required to find the minimum element.

Q47. Which of the following correctly declares a jagged array in Java?

- A. `int arr[][] = new int[3][3];`
- B. `int arr[][] = new int[3][];`
- C. `int arr[][] = new int[][3];`
- D. `int arr[3][] = new int[3][3];`

✓ **Answer:** B. `int arr[][] = new int[3][];`

💡 **Explanation:**

Jagged arrays have varying column lengths.

Q48. What happens if you try to access an invalid index in an array?

- A. `NullPointerException`
- B. `ArrayIndexOutOfBoundsException`
- C. `ClassCastException`
- D. No Exception

✓ **Answer:** B. `ArrayIndexOutOfBoundsException`

💡 **Explanation:**

Accessing an invalid index throws this exception.

Q49. What is the space complexity of an array of size n ?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

✓ **Answer:** C. $O(n)$

💡 **Explanation:**

An array of size n requires $O(n)$ space.

Q50. Which data structure is best suited for implementing a sparse matrix?

- A. 1D Array
- B. 2D Array
- C. Linked List
- D. Hash Map

✓ **Answer:** C. Linked List

💡 **Explanation:**

Sparse matrices can be stored efficiently using linked lists.

📁 6. Array Address Calculation (10 Questions)

Q51. What is the formula to calculate the address of an element in a 1D array?

- A. $\text{Base} + \text{index} * \text{size}$
- B. $\text{Base} + \text{size} * (\text{index} + 1)$
- C. $\text{Base} + \text{index} + \text{size}$
- D. $\text{Base} * \text{index} * \text{size}$

✓ **Answer:** A. $\text{Base} + \text{index} * \text{size}$

💡 **Explanation:**

To calculate address: $\text{address} = \text{base} + (\text{index} * \text{size})$.

Q52. What is the address of the element `arr[4]` if the base address is 2000 and each element is 4 bytes?

- A. 2004
- B. 2016
- C. 2012
- D. 2020

✓ **Answer:** B. 2016

💡 **Explanation:**

$\text{Address} = 2000 + 4 * 4 = 2016$.

Q53. What is the formula for the address calculation in a 2D row-major order?

- A. $\text{Base} + (i * \text{col_count} + j) * \text{size}$
- B. $\text{Base} + (i + \text{col_count} * j) * \text{size}$
- C. $\text{Base} * (i + j) * \text{size}$
- D. $\text{Base} + i * j * \text{size}$

✓ **Answer:** A. $\text{Base} + (i * \text{col_count} + j) * \text{size}$

💡 **Explanation:**

Row-major order computes address using this formula.

Q54. What is the address of `arr[2][3]` if the base address is 1000, and element size is 2 bytes in row-major order with 5 columns?

- A. 1020
- B. 1016

- C. 1028
- D. 1032

✓ **Answer:** A. 1020

💡 **Explanation:**

$$1000 + (2 * 5 + 3) * 2 = 1020.$$

Q55. What is the address of `arr[3][4]` in a 2D array with column-major order?

- A. $\text{Base} + (j * \text{row_count} + i) * \text{size}$
- B. $\text{Base} + (i * \text{col_count} + j) * \text{size}$
- C. $\text{Base} + (i + j) * \text{size}$
- D. $\text{Base} + i * j * \text{size}$

✓ **Answer:** A. $\text{Base} + (j * \text{row_count} + i) * \text{size}$

💡 **Explanation:**

Column-major order calculates the address using this formula.

Q56. Which of the following is correct for calculating the address of a 2D array element in column-major order?

- A. $\text{Base} + (j * \text{row_count} + i) * \text{size}$
- B. $\text{Base} + (i * \text{col_count} + j) * \text{size}$
- C. $\text{Base} + i * j * \text{size}$
- D. $\text{Base} + (i + j) * \text{size}$

✓ **Answer:** A. $\text{Base} + (j * \text{row_count} + i) * \text{size}$

💡 **Explanation:**

Column-major order uses this formula.

Q57. What is the base address of `arr[0][0]`?

- A. 0
- B. Initial memory allocated for the array
- C. Address of last element
- D. Address of the array itself

✓ **Answer:** B. Initial memory allocated for the array

💡 **Explanation:**

Base address points to the start of the array.

Q58. What is the address of `arr[3][2]` if base address is 4000 and size of each element is 4 bytes with 4 columns in row-major order?

- A. 4048
- B. 4056
- C. 4072
- D. 4080

✓ **Answer:** B. 4056

💡 **Explanation:**

$$4000 + (3 * 4 + 2) * 4 = 4056.$$

Q59. Which of the following techniques is used in sparse matrix representation?

- A. Linked List
- B. 2D Array
- C. Row-major order
- D. Hash Map

✓ **Answer:** A. Linked List

💡 **Explanation:**

Sparse matrices are stored efficiently using linked lists.

Q60. What is the address of `arr[2][3]` if base address is 2000, element size is 2 bytes, 5 columns in row-major order?

- A. 2012
- B. 2020
- C. 2030
- D. 2040

✓ **Answer:** B. 2020

💡 **Explanation:**

$$\text{Address} = 2000 + (2 * 5 + 3) * 2 = 2020.$$

✚ 7. Infix to Postfix (10 Questions)

Q61. What is the postfix expression for the infix expression:

$A + B * C$

- A. $ABC*+$
- B. $AB+C*$
- C. $ABC+$
- D. $ABC+$

✓ **Answer:** A. $ABC*+$

💡 **Explanation:**

Multiplication is evaluated before addition.

Q62. What is the postfix expression for:

$(A + B) * (C - D)$

- A. $AB+CD-*$
- B. $A+B*CD-$
- C. $AB+CD-$
- D. $A+BC-D-$

✓ **Answer:** A. $AB+CD-*$

💡 **Explanation:**

Parentheses are evaluated first, followed by multiplication.

✓ **[Remaining 60 MCQs for Stack, Queue, Linked List, Recursion, and Infix Evaluation are coming up...]**

💡 **Would you like to continue with the next section?** 😊

40

You said:

continue with some output based questions

ChatGPT said:

Here's the **continuation** with **50 output-based MCQs in Java on Data Structures** with answers and detailed explanations:

🚀 8. Stack (10 Output-Based Questions)

Q61. What is the output of the following Java code?

```
java
CopyEdit
import java.util.Stack;
public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(10);
        stack.push(20);
        stack.push(30);
        System.out.println(stack.peek());
    }
}
```

- A. 10
- B. 20
- C. 30
- D. Stack is empty

✓ **Answer:** C. 30

💡 **Explanation:**

`peek()` returns the top element of the stack without removing it.

Q62. What is the output of the following code?

```
java
CopyEdit
import java.util.Stack;
public class Main {
    public static void main(String[] args) {
        Stack<Character> stack = new Stack<>();
        stack.push('A');
        stack.push('B');
        stack.pop();
        System.out.println(stack.peek());
    }
}
```

- A. A
- B. B
- C. Stack is empty
- D. NullPointerException

✓ **Answer:** A. A

💡 **Explanation:**

`pop()` removes the top element (B), so `peek()` returns A.

Q63. What is the output of the following code?

```
java
CopyEdit
import java.util.Stack;
public class Main {
    public static void main(String[] args) {
        Stack<String> stack = new Stack<>();
        stack.push("Java");
        stack.push("Python");
        stack.pop();
        stack.push("C++");
        System.out.println(stack.peek());
    }
}
```

- A. Java
- B. Python
- C. C++
- D. NullPointerException

✓ **Answer:** C. C++

💡 **Explanation:**

After popping Python, C++ is pushed and becomes the top element.

Q64. What is the output of the following code?

```
java
CopyEdit
import java.util.Stack;
public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(5);
        stack.push(10);
        System.out.println(stack.pop() + stack.pop());
    }
}
```

- A. 15
- B. 10
- C. 5
- D. EmptyStackException

✓ **Answer:** A. 15

💡 **Explanation:**

stack.pop() returns 10, and the next pop() returns 5. So, $10 + 5 = 15$.

Q65. What is the output of the following code?

```
java
CopyEdit
import java.util.Stack;
public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        System.out.println(stack.isEmpty());
    }
}
```

- A. true
- B. false
- C. NullPointerException
- D. EmptyStackException

✓ **Answer:** A. true

💡 **Explanation:**

`isEmpty()` checks if the stack is empty and returns `true`.

Q66. What is the output of the following code?

```
java
CopyEdit
import java.util.Stack;
public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(1);
        stack.push(2);
        stack.push(3);
        System.out.println(stack.search(2));
    }
}
```

- A. 1
- B. 2
- C. 3
- D. -1

✓ **Answer:** B. 2

💡 **Explanation:**

`search(2)` returns 2, since it is 2 positions from the top.

Q67. What happens if you try to pop an element from an empty stack?

```
java
CopyEdit
Stack<Integer> stack = new Stack<>();
stack.pop();
```

- A. Returns 0
- B. Throws EmptyStackException
- C. Returns -1
- D. Throws NullPointerException

✓ **Answer:** B. Throws EmptyStackException

💡 **Explanation:**

Popping from an empty stack throws `EmptyStackException`.

Q68. What is the output of the following code?

```
java
CopyEdit
Stack<Integer> stack = new Stack<>();
stack.push(100);
stack.push(200);
stack.clear();
System.out.println(stack.isEmpty());
```

- A. false
- B. true
- C. NullPointerException
- D. 0

✓ **Answer:** B. true

💡 **Explanation:**

`clear()` empties the stack, and `isEmpty()` returns `true`.

Q69. What is the output of this code?

```
java
CopyEdit
Stack<Integer> stack = new Stack<>();
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println(stack.indexOf(20));
```

- A. 1
- B. 2
- C. -1
- D. 0

✓ **Answer:** A. 1

💡 **Explanation:**

`indexOf()` returns the position of the element, with 0-based indexing.

Q70. What is the result of the following code?

```
java
CopyEdit
Stack<Integer> stack = new Stack<>();
stack.push(5);
stack.push(10);
stack.push(15);
System.out.println(stack.size());
```

- A.** 1
- B.** 2
- C.** 3
- D.** 0

✓ **Answer:** C. 3

💡 **Explanation:**

`stack.size()` returns the number of elements in the stack.

☐ 9. Queue (10 Output-Based Questions)

Q71. What is the output of the following Java code?

```
java
CopyEdit
import java.util.LinkedList;
import java.util.Queue;
public class Main {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList<>();
        queue.add(10);
        queue.add(20);
        queue.add(30);
        System.out.println(queue.poll());
    }
}
```

- A.** 10
- B.** 20
- C.** 30
- D.** NullPointerException

✓ **Answer:** A. 10

💡 **Explanation:**

`poll()` removes and returns the front element, which is 10.

Q72. What is the output of the following code?

```
java
CopyEdit
Queue<String> queue = new LinkedList<>();
queue.add("A");
queue.add("B");
queue.remove();
queue.add("C");
System.out.println(queue.peek());
```

A. A

B. B

C. C

D. NullPointerException

✓ **Answer:** B. B

💡 **Explanation:**

`remove()` removes A, and `peek()` returns the front element, B.

Q73. What will be the output of the following code?

```
java
CopyEdit
Queue<Integer> queue = new LinkedList<>();
queue.offer(5);
queue.offer(10);
queue.poll();
System.out.println(queue.peek());
```

A. 5

B. 10

C. NullPointerException

D. EmptyQueueException

✓ **Answer:** B. 10

💡 **Explanation:**

`poll()` removes 5, so `peek()` returns 10.

Q74. What is the output of the following code?

```
java
```



```
CopyEdit
Queue<Integer> queue = new LinkedList<>();
queue.offer(1);
queue.offer(2);
queue.offer(3);
queue.poll();
System.out.println(queue.size());
```

- A. 1
- B. 2
- C. 3
- D. 0

✓ **Answer:** B. 2

💡 **Explanation:**

`poll()` removes one element, leaving 2 elements in the queue.

Q75. What happens if you try to remove an element from an empty queue using `remove()`?

- A. Returns null
- B. Throws `NoSuchElementException`
- C. Returns -1
- D. Throws `NullPointerException`

✓ **Answer:** B. Throws `NoSuchElementException`

💡 **Explanation:**

`remove()` throws `NoSuchElementException` when the queue is empty.

Q76. What is the output of the following code?

```
java
CopyEdit
Queue<Integer> queue = new LinkedList<>();
queue.add(100);
queue.add(200);
queue.clear();
System.out.println(queue.isEmpty());
```

- A. false
- B. true
- C. `NullPointerException`
- D. 0

✓ **Answer:** B. true

💡 **Explanation:**

`clear()` empties the queue, so `isEmpty()` returns true.

Q77. What is the result of the following code?

```
java
CopyEdit
Queue<String> queue = new LinkedList<>();
queue.add("Hello");
queue.add("World");
System.out.println(queue.remove());
```

- A.** World
- B.** Hello
- C.** NullPointerException
- D.** NoSuchElementException

✓ **Answer:** B. Hello

💡 **Explanation:**

`remove()` removes and returns the front element.

Q78. What is the result of the following code?

```
java
CopyEdit
Queue<Integer> queue = new LinkedList<>();
System.out.println(queue.poll());
```

- A.** 0
- B.** -1
- C.** null
- D.** EmptyQueueException

✓ **Answer:** C. null

💡 **Explanation:**

`poll()` returns `null` if the queue is empty.

Q79. What is the output of this code?

```
java
CopyEdit
Queue<Integer> queue = new LinkedList<>();
queue.offer(5);
queue.offer(10);
queue.offer(15);
queue.clear();
System.out.println(queue.size());
```

- A. 0
- B. 1
- C. 2
- D. 3

✓ **Answer:** A. 0

💡 **Explanation:**

`clear()` empties the queue, so size becomes 0.

Q80. What is the output of the following code?

```
java
CopyEdit
Queue<Integer> queue = new LinkedList<>();
queue.offer(5);
queue.offer(10);
queue.offer(15);
System.out.println(queue.contains(10));
```

- A. true
- B. false
- C. NullPointerException
- D. NoSuchElementException

✓ **Answer:** A. true

💡 **Explanation:**

`contains()` returns `true` if the element is present.

✓ [Next Sections: Linked List, Recursion, Infix to Postfix, and Infix Evaluation with 40 more output-based MCQs are coming up!]

💡 Do you want to proceed with the next sections? 😊

You said:

continue

ChatGPT said:

Here's the **continuation** with the next **40 output-based MCQs** on **Linked List, Recursion, Infix to Postfix, and Infix Evaluation** with answers and explanations:

🔗 10. Linked List (10 Output-Based Questions)

Q81. What is the output of the following Java code?

```
java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        System.out.println(head.next.data);
    }
}
```

- A. 10
- B. 20
- C. 30
- D. NullPointerException

✓ **Answer:** B. 20

💡 **Explanation:**

`head.next` points to the second node, which holds the value 20.

Q82. What is the output of the following code?

```
java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static void main(String[] args) {
        Node head = new Node(5);
        head.next = new Node(15);
        head.next.next = new Node(25);
        head.next.next.next = head;
        System.out.println(head.next.next.next.data);
    }
}
```

```
    }  
}
```

- A. 5
- B. 15
- C. 25
- D. NullPointerException

✓ **Answer:** A. 5

💡 **Explanation:**

`head.next.next.next` points back to the head (circular list).

Q83. What is the output of this code?

```
java  
CopyEdit  
class Node {  
    int data;  
    Node next;  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Node head = new Node(1);  
        head.next = new Node(2);  
        head.next.next = null;  
        System.out.println(head.next.next);  
    }  
}
```

- A. 0
- B. null
- C. 1
- D. 2

✓ **Answer:** B. null

💡 **Explanation:**

`head.next.next` is explicitly set to `null`.

Q84. What is the output of the following Java code?

```
java  
CopyEdit  
class Node {  
    int data;  
    Node next;
```

```

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static Node insertAtEnd(Node head, int data) {
        Node newNode = new Node(data);
        if (head == null) return newNode;
        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }
        temp.next = newNode;
        return head;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head = insertAtEnd(head, 20);
        System.out.println(head.next.data);
    }
}

```

- A. 10
- B. 20
- C. 0
- D. NullPointerException

✓ **Answer:** B. 20

💡 **Explanation:**

insertAtEnd() adds 20 at the end, so head.next.data becomes 20.

Q85. What will be the output of the following code?

```

java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static int countNodes(Node head) {
        int count = 0;
        while (head != null) {
            count++;
            head = head.next;
        }
        return count;
    }
}

```

```

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        System.out.println(countNodes(head));
    }
}

```

- A. 1
- B. 2
- C. 3
- D. 0

✓ **Answer:** C. 3

💡 **Explanation:**

There are 3 nodes in the list, so `countNodes()` returns 3.

Q86. What is the output of this code?

```

java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static Node deleteHead(Node head) {
        if (head == null) return null;
        return head.next;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head = deleteHead(head);
        System.out.println(head.data);
    }
}

```

- A. 10
- B. 20
- C. null
- D. 0

✓ **Answer:** B. 20

💡 **Explanation:**

`deleteHead()` removes the first node, making 20 the new head.

Q87. What is the output of the following Java code?

```
java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static Node reverse(Node head) {
        Node prev = null;
        Node curr = head;
        while (curr != null) {
            Node next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }

    public static void main(String[] args) {
        Node head = new Node(1);
        head.next = new Node(2);
        head.next.next = new Node(3);
        head = reverse(head);
        System.out.println(head.data);
    }
}
```

- A. 1
- B. 2
- C. 3
- D. NullPointerException

✓ **Answer:** C. 3

💡 **Explanation:**

The list is reversed, and the new head is 3.

Q88. What is the output of this code?

```
java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
```



```

        this.next = null;
    }
}

public class Main {
    public static Node deleteTail(Node head) {
        if (head == null || head.next == null) return null;
        Node temp = head;
        while (temp.next.next != null) {
            temp = temp.next;
        }
        temp.next = null;
        return head;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head = deleteTail(head);
        System.out.println(head.next);
    }
}

```

- A. null
- B. 10
- C. 20
- D. NullPointerException

✓ **Answer:** A. null

💡 **Explanation:**

`deleteTail()` removes the last node, making `head.next` null.

Q89. What is the result of the following code?

```

java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static int sumList(Node head) {
        int sum = 0;
        while (head != null) {
            sum += head.data;
            head = head.next;
        }
        return sum;
    }

    public static void main(String[] args) {

```

```

        Node head = new Node(1);
        head.next = new Node(2);
        head.next.next = new Node(3);
        System.out.println(sumList(head));
    }
}

```

- A. 1
- B. 3
- C. 6
- D. 5

✓ **Answer:** C. 6

💡 **Explanation:**

Sum of all elements is $1 + 2 + 3 = 6$.

Q90. What is the result of the following code?

```

java
CopyEdit
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {
    public static int findMiddle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow.data;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);
        System.out.println(findMiddle(head));
    }
}

```

- A. 10
- B. 20
- C. 30
- D. 40

✓ **Answer:** C. 30

💡 **Explanation:**

Middle element in a list of 5 elements is 30.

🔄 11. Recursion (10 Output-Based Questions)

Q91. What is the output of the following code?

```
java
CopyEdit
public class Main {
    public static int factorial(int n) {
        if (n == 0) return 1;
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

- A. 24
- B. 120
- C. 60
- D. 15

✓ **Answer:** B. 120

💡 **Explanation:**

Factorial of 5 = $5 * 4 * 3 * 2 * 1 = 120$.

Q92. What is the output of this code?

```
java
CopyEdit
public class Main {
    public static int sum(int n) {
        if (n == 0) return 0;
        return n + sum(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(sum(5));
    }
}
```

- A. 5
- B. 10
- C. 15
- D. 0

✓ **Answer:** C. 15

💡 **Explanation:**

Sum of numbers from 1 to 5 = 1 + 2 + 3 + 4 + 5 = 15.

Q93. What is the output of this code?

```
java
CopyEdit
public class Main {
    public static int fibonacci(int n) {
        if (n <= 1) return n;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        System.out.println(fibonacci(6));
    }
}
```

- A. 5
- B. 8
- C. 13
- D. 21

✓ **Answer:** B. 8

💡 **Explanation:**

Fibonacci series: 0, 1, 1, 2, 3, 5, 8.

Q94. What is the output of this code?

```
java
CopyEdit
public class Main {
    public static void printReverse(int n) {
        if (n == 0) return;
        System.out.print(n + " ");
        printReverse(n - 1);
    }

    public static void main(String[] args) {
        printReverse(5);
    }
}
```

- A. 5 4 3 2 1
- B. 5 4 3 2 1 0
- C. 0 1 2 3 4 5
- D. 1 2 3 4 5

✓ **Answer:** A. 5 4 3 2 1

💡 **Explanation:**

Prints numbers in reverse order using recursion.

Q95. What is the result of the following code?

```
java
CopyEdit
public class Main {
    public static int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }

    public static void main(String[] args) {
        System.out.println(gcd(48, 18));
    }
}
```

- A. 6
- B. 12
- C. 18
- D. 24

✓ **Answer:** A. 6

💡 **Explanation:**

GCD of 48 and 18 is 6 using the Euclidean algorithm.

✓ **[Coming Up: 20 More MCQs on Infix to Postfix and Infix Evaluation with Detailed Explanations]**

💡 **Do you want to continue?** 😊

You said:

continue

ChatGPT said:

Here's the **continuation** with the final **20 output-based MCQs on Infix to Postfix and Infix Evaluation** with answers and explanations:

12. Infix to Postfix (10 Output-Based Questions)

Q96. What is the output of the following Java code?

```
java
CopyEdit
import java.util.Stack;

public class Main {
    public static int precedence(char ch) {
        switch (ch) {
            case '+':
            case '-':
                return 1;
            case '*':
            case '/':
                return 2;
            case '^':
                return 3;
        }
        return -1;
    }

    public static String infixToPostfix(String exp) {
        StringBuilder result = new StringBuilder();
        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < exp.length(); i++) {
            char c = exp.charAt(i);

            if (Character.isLetterOrDigit(c)) {
                result.append(c);
            } else if (c == '(') {
                stack.push(c);
            } else if (c == ')') {
                while (!stack.isEmpty() && stack.peek() != '(') {
                    result.append(stack.pop());
                }
                stack.pop();
            } else {
                while (!stack.isEmpty() && precedence(c) <=
precedence(stack.peek())) {
                    result.append(stack.pop());
                }
                stack.push(c);
            }
        }

        while (!stack.isEmpty()) {
            result.append(stack.pop());
        }

        return result.toString();
    }
}
```

```

    }

    public static void main(String[] args) {
        String exp = "a+b*(c^d-e)^(f+g*h)-i";
        System.out.println(infixToPostfix(exp));
    }
}

```

- A. $abcd^e-fgh^{*+^+i-}$
- B. $abcd^e-fgh^{++i-}$
- C. $abc^de-fgh^{+^+i-}$
- D. $abc^d-e+fgh^{*+^+i-}$

✓ **Answer:** C. $abc^de-fgh^{+^+i-}$

💡 **Explanation:**

The given expression is converted into postfix notation correctly following precedence rules.

Q97. What is the result of converting the infix expression $A * (B + C) / D$ to postfix?

- A. $ABC+D/$
- B. $ABC+/D$
- C. $ABC+D/*$
- D. $A*B+C/D$

✓ **Answer:** A. $ABC+*D/$

💡 **Explanation:**

- $B + C \rightarrow BC +$
 - $A * (B + C) \rightarrow ABC + *$
 - $ABC + * / D \rightarrow ABC + * D /$
-

Q98. What is the postfix form of the expression $A + B * C - D / E$?

- A. $ABC^{*+}DE/-$
- B. $AB+CDE/-$
- C. $ABCDE/-+$
- D. $ABC^{*+}DE/-$

✓ **Answer:** A. $ABC^{*+}DE/-$

💡 **Explanation:**

- $B * C \rightarrow BC *$
- $A + BC * \rightarrow ABC^{*+}$
- $D / E \rightarrow DE /$
- $ABC^{*+}DE / -$

Q99. What is the postfix of the infix expression $A * (B + C * D) + E$?

- A. $ABCD*+E+$
- B. $AB+CD+E*$
- C. $ABC+DE+$
- D. $ABCD+*E+$

✓ **Answer:** A. $ABCD*+*E+$

💡 **Explanation:**

- $C * D \rightarrow CD*$
- $B + CD* \rightarrow BCD*+$
- $A * (B + CD*) \rightarrow ABCD*+*$
- $ABCD*+*E+$

Q100. Convert the expression $A + B * (C - D) / E$ to postfix.

- A. $ABCD-E/+$
- B. $AB+CD-E/$
- C. $ABC+DE/+$
- D. $ABCD-E/+$

✓ **Answer:** A. $ABCD-*E/+$

💡 **Explanation:**

- $C - D \rightarrow CD-$
- $B * (C - D) \rightarrow BCD-*$
- $BCD-* / E \rightarrow ABCD-*E/+$

Q101. What is the postfix form of $A * (B + C * D) / E + F$?

- A. $ABCD*+E/F+$
- B. $ABCD+E/F+$
- C. $ABC+DE/+F$
- D. $ABC*D+E/F+$

✓ **Answer:** A. $ABCD*+*E/F+$

💡 **Explanation:**

- $C * D \rightarrow CD*$
- $B + CD* \rightarrow BCD*+$
- $A * (B + CD*) \rightarrow ABCD*+*$
- $ABCD*+* / E \rightarrow ABCD*+*E/$

- $ABCD^{*+*E}/F+$

Q102. What is the result of converting $((A+B) * C) - (D / (E+F))$ to postfix?

- A. $AB+CDEF+/-$
- B. $AB+CDEF+/-$
- C. $ABC+DEF/-$
- D. $ABCDEF/-+$

✓ **Answer:** A. $AB+C*DEF+/-$

💡 **Explanation:**

- $A+B \rightarrow AB+$
- $(A+B) * C \rightarrow AB+C*$
- $E+F \rightarrow EF+$
- $D / (E+F) \rightarrow DEF+ /$
- $AB+C* - DEF+ /$

Q103. Convert $A+B * (C-D) / E-F$ to postfix.

- A. $ABCD-E/+F-$
- B. $AB+CD-E/F-$
- C. $ABCD-E/+F-$
- D. $ABCD+E/F-$

✓ **Answer:** A. $ABCD-*E/+F-$

💡 **Explanation:**

- $C-D \rightarrow CD-$
- $B * (C-D) \rightarrow BCD-*$
- $BCD-* / E \rightarrow ABCD-*E/+$
- $ABCD-*E/+ - F$

Q104. What is the postfix of $A * (B+C * (D/E))$?

- A. $ABCDE/+$
- B. $ABCDE/+$
- C. $ABCDE/+$
- D. $ABCDE+/*$

✓ **Answer:** A. $ABCDE/+$

💡 **Explanation:**

- $D/E \rightarrow DE/$
- $C * (D/E) \rightarrow CDE/*$
- $B + C * (D/E) \rightarrow BCDE/*+$
- $A * (B + C * (D/E)) \rightarrow ABCDE/*+*$

Q105. Convert the infix expression $A+B-C * (D/E)$ to postfix.

- A. $AB+CDE/-$
- B. $ABC+DE/-$
- C. $ABCDE/-$
- D. $ABCDE/-+$

✓ **Answer:** A. $AB+CDE/*-$

💡 **Explanation:**

- $D/E \rightarrow DE/$
- $C * (D/E) \rightarrow CDE/*$
- $A+B \rightarrow AB+$
- $AB+CDE/*-$

13. Evaluation of Infix/Postfix (10 Output-Based Questions)

Q106. Evaluate the postfix expression $5\ 3\ 2\ *\ +\ 4\ -$.

- A. 5
- B. 7
- C. 9
- D. 3

✓ **Answer:** B. 7

💡 **Explanation:**

- $3 * 2 = 6$
- $5 + 6 = 11$
- $11 - 4 = 7$

Q107. Evaluate the postfix expression $10\ 2\ 8\ *\ +\ 3\ -$.

- A. 23
- B. 18
- C. 21
- D. 25

✓ **Answer:** C. 21

💡 **Explanation:**

- $2 * 8 = 16$
 - $10 + 16 = 26$
 - $26 - 3 = 21$
-

Q108. Evaluate the postfix expression $5\ 6\ 2\ *\ +\ 9\ -$.

- A. 12
- B. 3
- C. 8
- D. 7

✓ **Answer:** B. 3

💡 **Explanation:**

- $6 * 2 = 12$
 - $5 + 12 = 17$
 - $17 - 9 = 8$
-

Q109. Evaluate the postfix expression $7\ 2\ 3\ *\ -\ 4\ /\$.

- A. 1
- B. 2
- C. 3
- D. 0

✓ **Answer:** D. 0

💡 **Explanation:**

- $2 * 3 = 6$
 - $7 - 6 = 1$
 - $1 / 4 = 0$ (integer division)
-

Q110. Evaluate the postfix expression $12\ 4\ 3\ *\ +\ 8\ /\$.

- A. 5
- B. 6
- C. 4
- D. 7

✓ **Answer:** A. 5

💡 **Explanation:**

- $4 * 3 = 12$
 - $12 + 12 = 24$
 - $24 / 8 = 3$
-

Q111. Evaluate the postfix expression $8\ 3\ 2\ *\ +\ 4\ -$.

- A. 6
- B. 7
- C. 8
- D. 9

✓ **Answer:** D. 9

💡 **Explanation:**

- $3 * 2 = 6$
 - $8 + 6 = 14$
 - $14 - 4 = 9$
-

Q112. Evaluate the postfix expression $5\ 9\ 3\ +\ 4\ *\ -$.

- A. -31
- B. -40
- C. -37
- D. -29

✓ **Answer:** C. -37

💡 **Explanation:**

- $9 + 3 = 12$
 - $12 * 4 = 48$
 - $5 - 48 = -37$
-

Q113. Evaluate the postfix expression $7\ 4\ 5\ +\ *\ 2\ -$.

- A. 61
- B. 60
- C. 55
- D. 62

✓ **Answer:** A. 61

💡 **Explanation:**

- $4 + 5 = 9$
 - $7 * 9 = 63$
 - $63 - 2 = 61$
-

Q114. Evaluate the postfix expression $10\ 2\ 8\ *\ +\ 3\ /\$.

- A. 8
- B. 10
- C. 14
- D. 9

✓ **Answer:** A. 8

💡 **Explanation:**

- $2 * 8 = 16$
 - $10 + 16 = 26$
 - $26 / 3 = 8$ (integer division)
-

Q115. Evaluate the postfix expression $15\ 7\ 1\ 1\ +\ -\ /\ 3\ * 2\ 1\ 1\ +\ +\ -$.

- A. 5
- B. 10
- C. -4
- D. 6

✓ **Answer:** C. -4

💡 **Explanation:**

- $1 + 1 = 2$
- $7 - 2 = 5$
- $15 / 5 = 3$
- $3 * 3 = 9$
- $1 + 1 = 2$
- $9 - 2 = -4$